

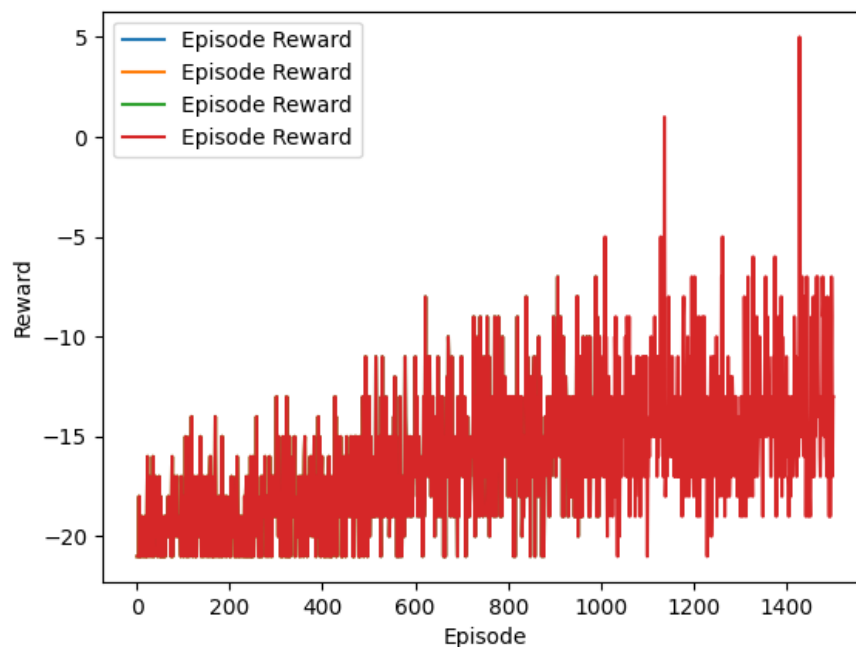
HW1-Part2-Problem2

Please note that training on this model was interrupted several times due to the use of jupyter notebook (which we were allowed to use). Jupyter notebook has two problems: doesn't seem to actually execute code on GPU and also it is subject to autokill every 1 hour as set by Mr. Mateusz.

Run 1:

(refer to folder: Run1 _ Interrupted)

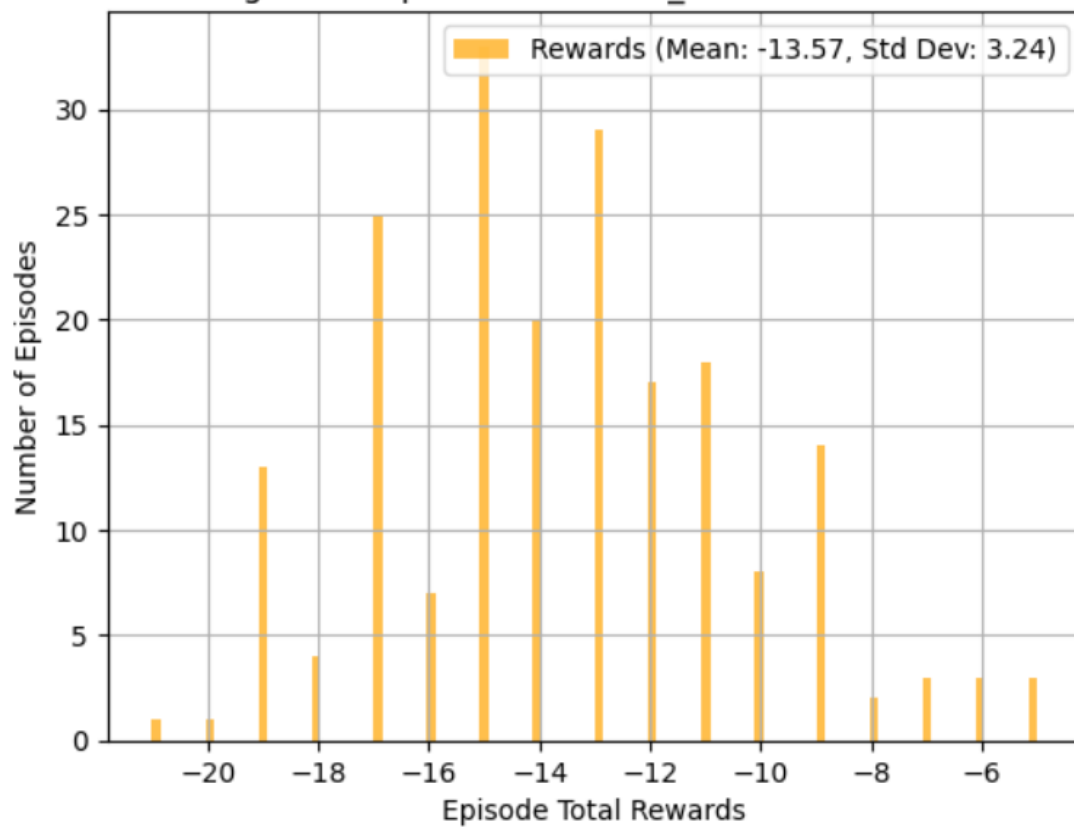
The first successful run1 was interrupted after ~ 30 hours (1736 episodes) (noticed interruption on Oct 23 evening). Unfortunately, only the policy model up to 1500 episodes was saved and only the plot of episode rewards, at episode 1500 was saved. Here is the plot.

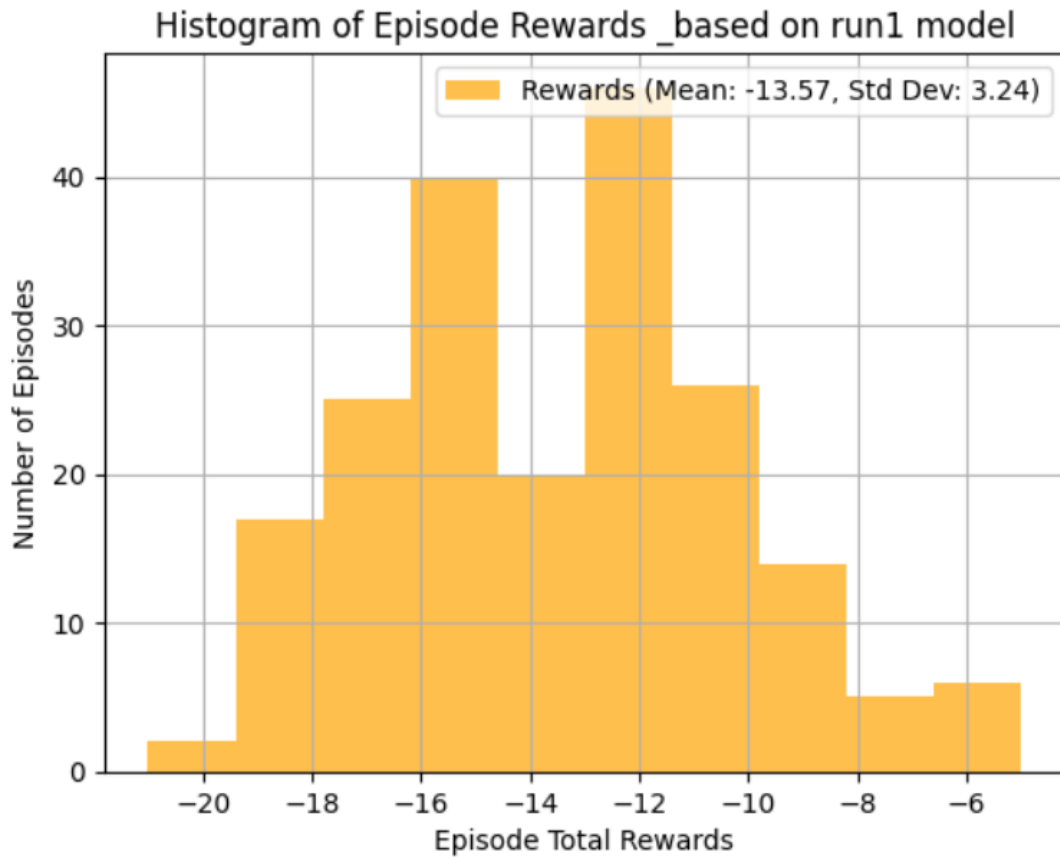


The plot shows obvious learning trend, and less variance than what we're used to without the non-constant baseline.

Then we use the trained model of run 1 (policy_model_1500) to simulate 200 episodes, and guide the actions in each state based on the trained policy. The histogram based on run1:

Histogram of Episode Rewards _based on run1 model

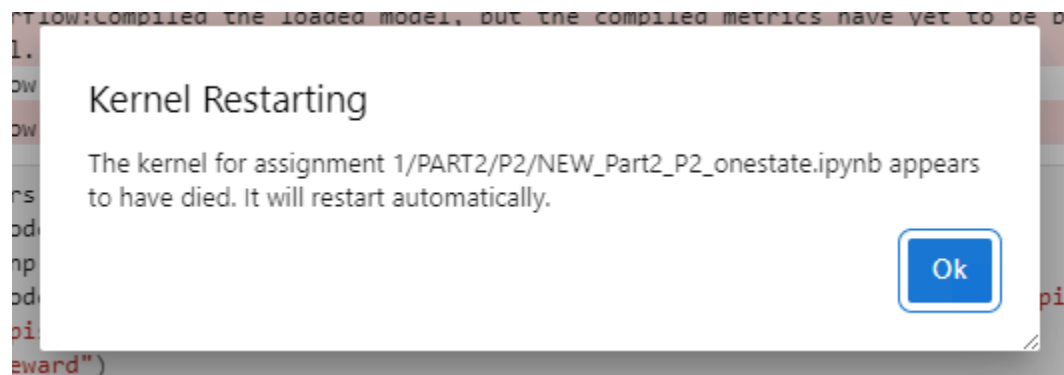


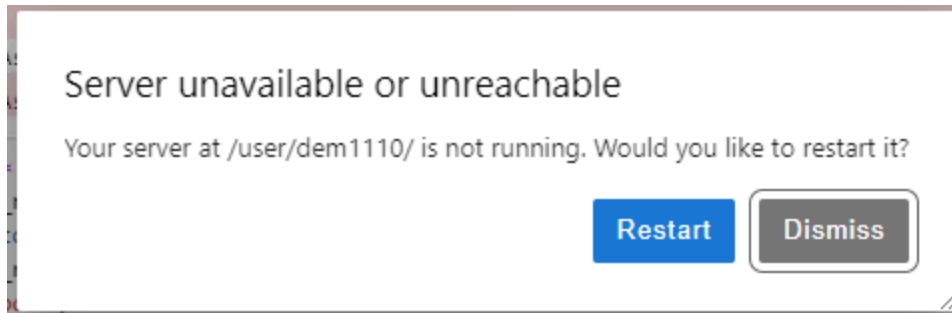


Run2:

(Refer to folder: Run 2 _ Interrupted)

This run was initiated on Oct 23 in the evening. However, it was interrupted due to a server restart (autokill), after about 300~400 episodes at night.

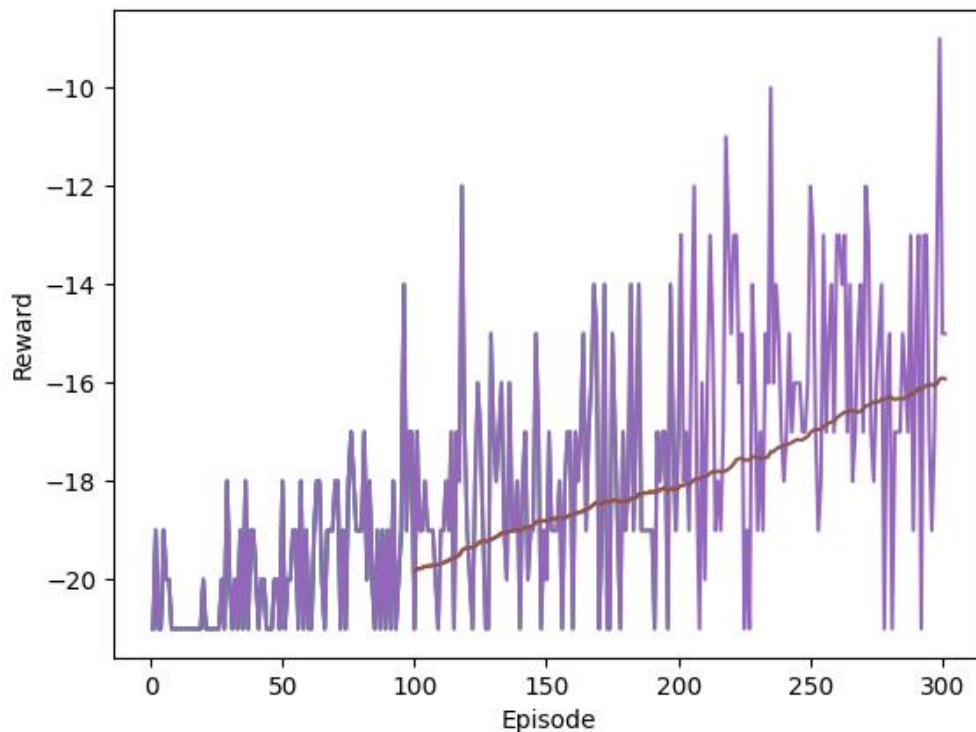




Unfortunately, only the model and plots at episode 300 were saved, and NumPy array of the episode rewards.

The best plot of the episodes' rewards versus rewards from that run, at episode 300. This plot shows the episodes' rewards for each episode along with the moving average over the last 100 episodes. This plot clearly shows a learning trend, and lower variance than when using policy gradient without non-constant baseline.

(the legend was removed on purpose).



Run3:

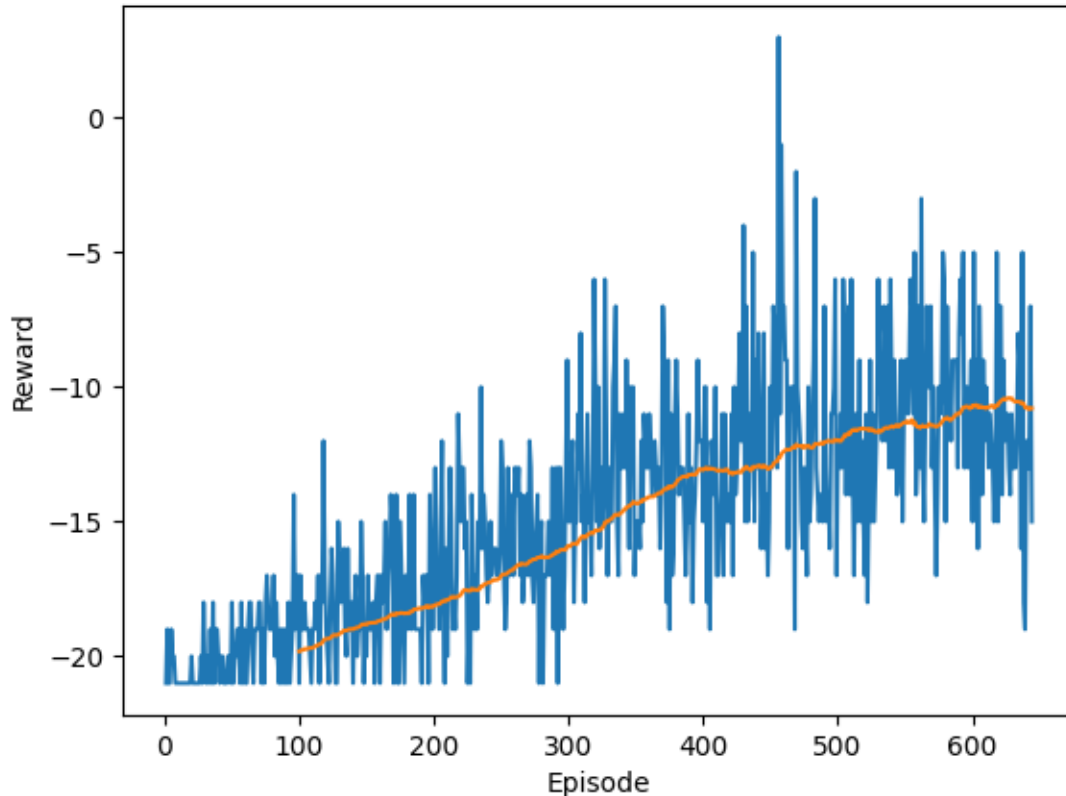
(Refer to folder: Run 3 _ proceeded from run 2)

Using the saved models of run 2 at episode 300, and the NumPy array of episode rewards up to episode 300, we load both files and continue training the model. Run 3 is initiated in the morning-noon of October 24 due to the interruption of the server.

We start from batch =300. However, this code was interrupted paused twice to change the printing criteria & saving criteria of the model. It was paused once after ~70-80 episodes, and another time after less episodes. There was no model saved during these interruptions, however the kernel was still active so the model kept learning and updating and the list of episode rewards was updated. The final run is run 4, when the code was still training from run 3 without any interruptions. For this reason in the folder you will find that the plots of episode rewards that refer to about 500 episodes are labeled 400, and same for models and arrays. This is just due to the number of the episode after the pause which was set to 300 again instead of ~400~420.

You will find two codes in the folder. Run4 is the final one. You will also find the plots and episode rewards of this run along with the best value and policy models.

The best plots of episode rewards and the moving average across 100 episodes at this run is show below.



Testing the trained model

In folder “Testing on 400 episodes”, you will find the code that loads the trained model and tests it on 500 episodes, by selecting an action based on the trained policy.

Note that due to the limitation of time, and trouble with resource allocation on deepdish and inability to access GPU; not the final trained model was used, rather the trained model at approximately episode 400. For this exercise, we only need the trained model of the policy, since the value model is only used in the loss function to train the policy better.

We load the trained model (also present in the same folder), `policy_model_400`.

And then we ran the simulation of 500 episodes.

The following histogram of episode rewards is obtained with the values of mean and standard deviation.

Histogram of Episode Rewards _based on run4 model

